Bnot Elisheva High School

Final Project in Electronics and Computers

Specialization: Computer Systems

A Shopping Cart that Follows the Shopper

Submitted by:

Aviya Hochman

Esti Toledano

Supervised by: Shira Nachmani and Efrat Trabelsi

Academic year 2023–2024

Table of Contents

Students' Declaration....3
Background....4
Block Diagram....5
Electrical Schematic....6
Explanation of System Operation....8
Project Documentation – Component Details
ARDUINO UNO Controller....9
Ultrasonic Distance Sensor....10
IR Sensor....13
DC Motor....15
Motor Driver....17
Color Graphic LCD....20
Setting the Color of Each Pixel....21
Touch Screen....22
Explanation of Screen Functions....23
Screen Documentation....29
Measurements (Distance Sensor and Screen)....32
Flowchart....34
Program Segment....35
Log....39
Faults....42
SPI Communication....43
Communication Protocol....44
Summary....45

Students' Declaration

Aviya Hochman

Esti Toledano

I/we, the undersigned, hereby declare that the final project and the project book were done by me/us alone.

The final project was done on the basis of the subjects I studied at school and independently.

The final project and the project book were done on the basis of the guidance of the personal supervisor.

The sources of information I used to carry out the final project are listed in the list of sources in the project book.

I am aware of the responsibility I take upon myself by signing this declaration, that everything stated in it is true and only the truth.

Background

As part of the Electronics and Computer Engineering track classes, we were asked to prepare a project that summarizes the subjects that were studied during the years of study in the track.

As is known, the role of technological progress in general, and of electronics practical engineers in particular, is to ease and help humanity by developing new devices for the market.

We thought of a device that would ease people's everyday life.

People go shopping on a regular basis, and the existing shopping cart has several drawbacks. Because of the weight of the products, maneuvering it is complicated, and therefore we created a shopping cart that will follow the shopper.

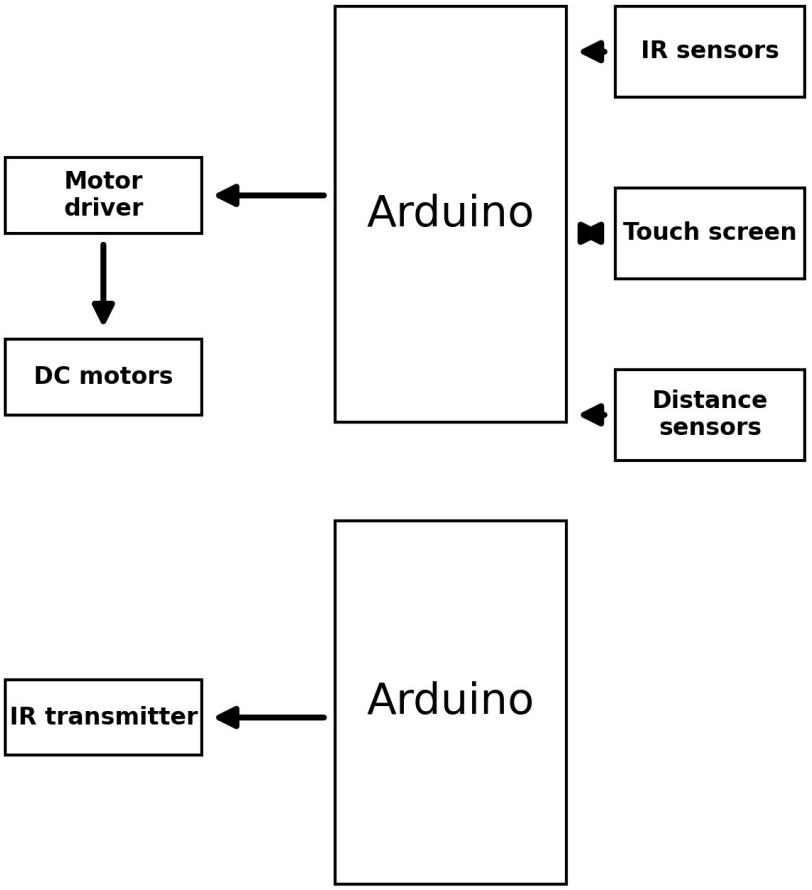
Technical specification

- 2 Arduinos
- IR sensors for following the shopper
- IR transmitter
- Distance sensor for collision prevention
- Touch screen
- DC motors for pushing the cart
- d293l motor driver

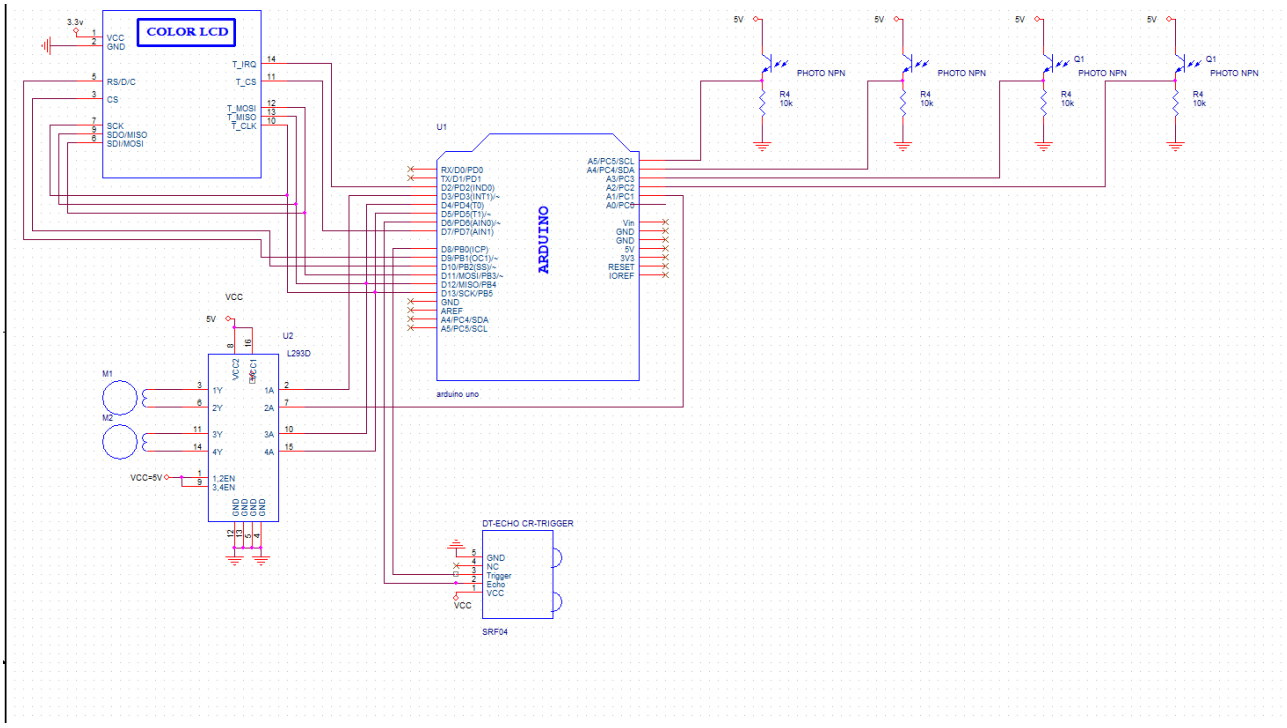
Technologies

SPI protocol for the color touch screen and for the Voicer.

Diagram


Arduino
IR sensors
Touch screen
Distance sensors
Motor driver
DC motors
Arduino
IR transmitter

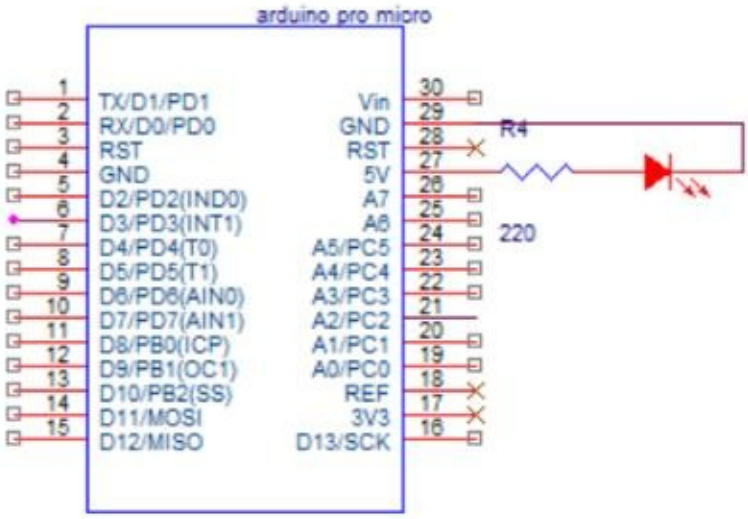
Electrical Schematic



Arduino Pro Micro

We used the Pro Micro in order to supply voltage to an infrared LED that will be received by the phototransistors in the Arduino Uno.

LED connection



Explanation of the System Components' Operation:

Arduino Uno:

This is the brain of the system. The Arduino Uno is the controller; it receives commands and executes them. Connected to it are IR sensors, a distance sensor, DC motors and a touch screen. All the software is burned into the controller and it manages the entire system, also by receiving signals from the Arduino Nano.

Graphic display and touch screen:

A color graphic display of 240 x 320 pixels; the shopper's press options appear on the display during the shopping.

Arduino Pro Micro:

This is a microcontroller to which an LED that transmits infrared light to the IR sensors is connected.

IR sensor:

This is a sensor that receives infrared light and outputs 1 or 0 according to the presence or absence of light within a range of about 5 cm. We connected 4 sensors, one on each side of the cart, so that the cart will identify in which direction the shopper is standing. The sensors are connected to the Arduino Uno.

Distance sensor:

This is a sensor that sends sound waves and detects the distance from an object standing in front of it according to the time it took the wave to return to the sensor. This sensor is connected to the Arduino Uno and we used it to check throughout the program that the cart is not about to get stuck, and if it detects an object in front of it, it will perform a bypass.

DC motors:

The purpose of the DC motors is to turn the cart's wheels according to the instructions that will be sent from the program, based on the presses and the data sent to it. The motors are connected to the Arduino Uno.

Infrared LED:

This is a component made of a diode. The diode transmits infrared light when voltage is supplied to it. It is connected to the Pro Micro and the cart drives according to it – it is with the shopper.

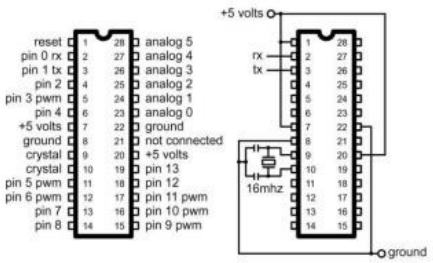
Project Documentation and Component Details

In our project there is an Arduino Uno and a Pro Micro. To the Arduino Uno we connected all the components except the LED, and the connection between it and the Arduino Pro Micro is by receiving, in the Arduino Uno, the light that is transmitted from the Pro Micro.

ARDUINO UNO Controller

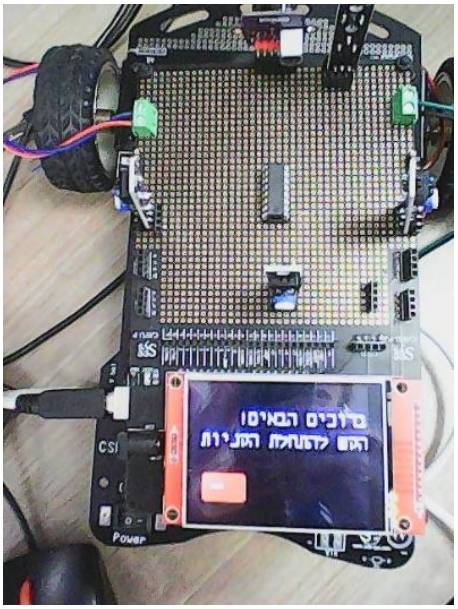
Based on the Atmega328p microcontroller

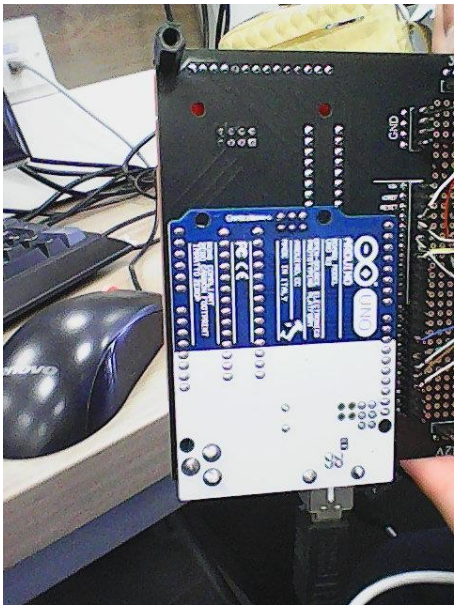




The main features of the Atmega328p component are:

- 16MHZ clock frequency.
- 5V operating voltage (supply voltage to the board 7V-12V).
- Current at the I/O pins up to 40mA.
- Program memory (flash) of 32k.
- Data memory (RAM) of 2k.
- 14 digital inputs and outputs.
- 6 PWM outputs.
- 6 analog inputs with a resolution of 10 bits.
- Serial communication (rs232, i2c, spi).
- 2 external interrupts.





Ultra-sonic SRF04 Distance Sensor

After the program is activated by pressing screen 1 (detailed later), the programs of the distance sensor and of the IR sensors are activated. The distance sensor program receives the distance, and if it is too short it performs a bypass.

Component description:

To measure the distance, we measure the 'time' and use the speed formula: **V = S / t**

Where: V – speed. S – distance (in units of meter). t – time (in units of second).

The microcontroller measures the time in microseconds. In order to get the distance in cm, we divide the time that the sensor measured by the number 58.

Explanation:

It is known that the speed of sound waves (at 24 °C) is: 346 meter/second. That is, the **propagation time** (denoted by X) for **one cm** is:

X = 2.89 milli second / meter = 28.9 micro second / cm

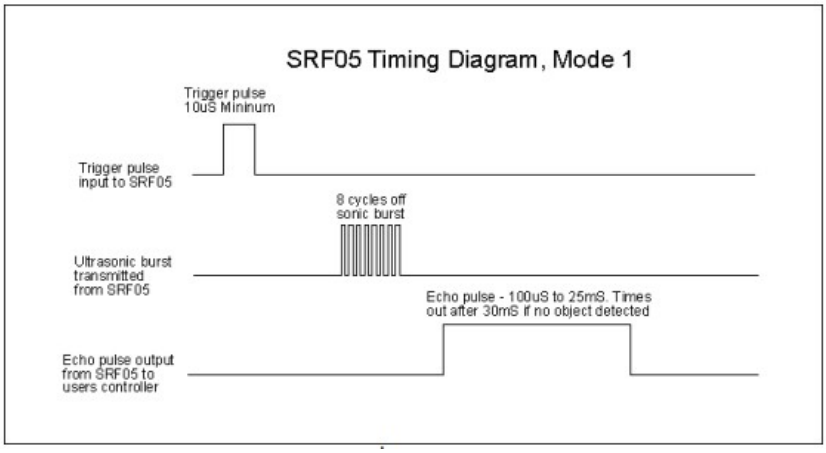
The 'measured' distance is: S = t / x = t / 28.9 (S – distance in cm, t – time in uS, X – propagation time of sound waves in uS/cm).

The sound waves travel a double path (from the sensor to the surface and back), therefore the time must be divided by 2. That is:

S = (t/2) / x = (t/2) / 28.9

Therefore, in order to get the distance (S), the 'measured time' (t) must be divided by the number: **28.9 · 2 ≅ 58**.

Image of the waves at the sensor's pins



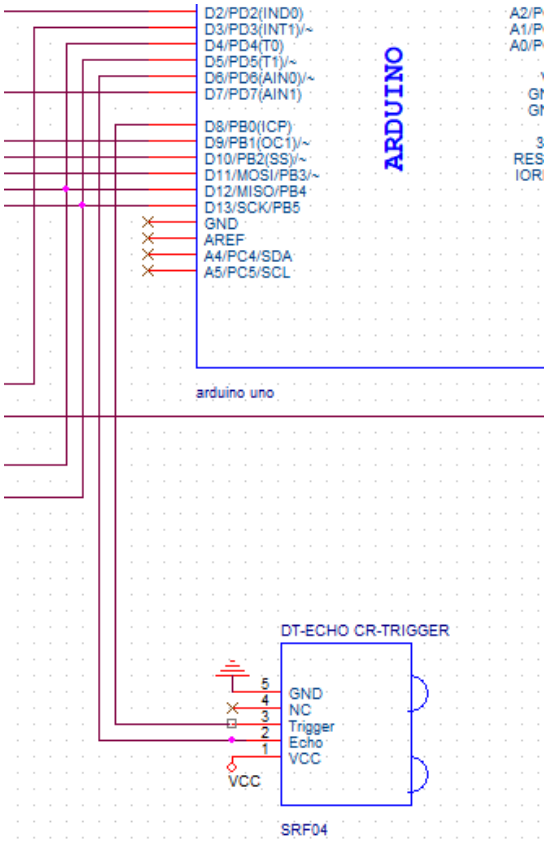
Program for activating the distance sensor:

```
int trig=8,echo=6;
void setup() {
  pinMode(trig,OUTPUT);
  pinMode(echo,INPUT);
  Serial.begin(9600);

}
int time,distance;

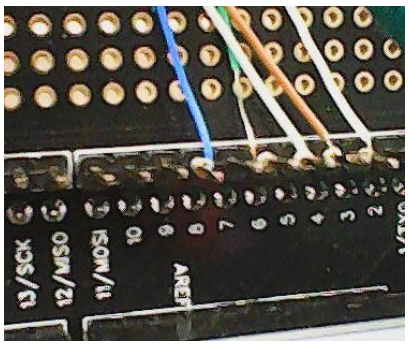
void loop() {
  distance=0;
  time=0;
  digitalWrite(trig,HIGH);//trigger
  delayMicroseconds(10);
  digitalWrite(trig,LOW);//trigger
  while(digitalRead(echo)==0);//echo
  time=micros();//micro second time
  while(digitalRead(echo)==1);//echo
  time=micros()-time;
  distance=time/58;
  Serial.println(distance);
  delay(1500);
}
```

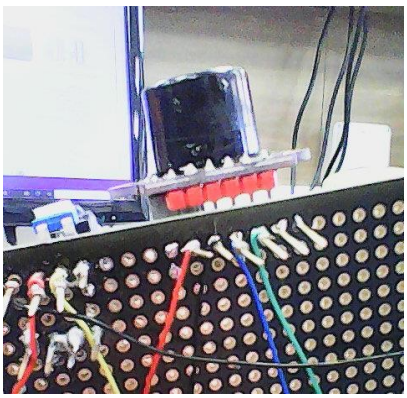
Connections to the distance sensor

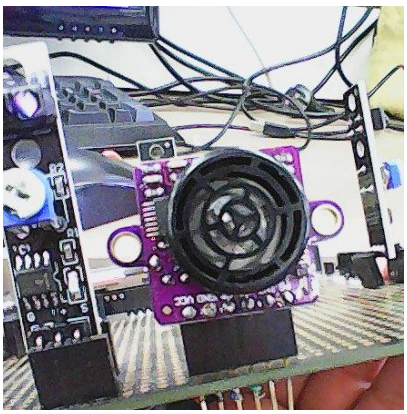


Distance Sensor Connections

We connected 4 wires to the distance sensor: the first to the voltage output – in order to supply voltage to the sensor, the second to ground, the third is connected from the echo pin of the sensor, the pin that changes from 1 to 0 on the output and return of the sound wave, to pin D6 of the Arduino Uno. The fourth wire is connected from the trigger pin of the distance sensor to pin D8 of the Arduino Uno; through this pin the Arduino transmits a pulse to the sensor.



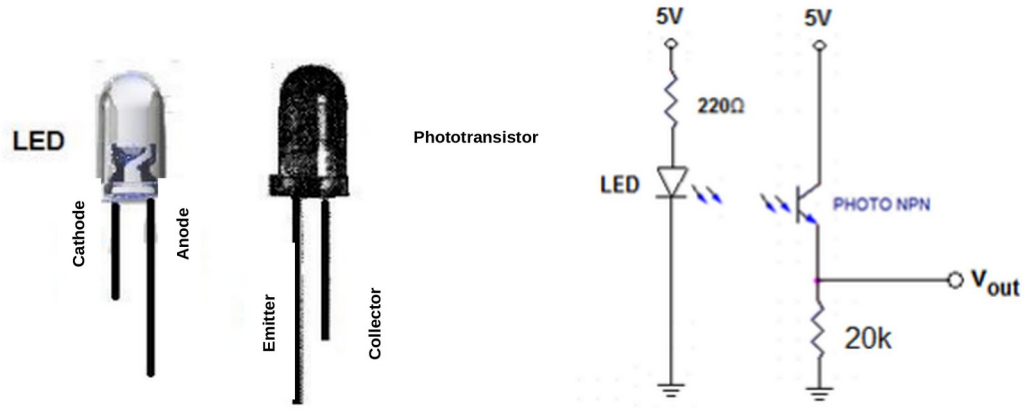




Measurements on the sensor at the end of the book.

IR Sensor (Infra-Red)

The sensor is built from an LED that radiates IR light and a phototransistor that is sensitive to IR light. The current in the emitter of the phototransistor depends on the intensity of the light at the base of the transistor. The greater the light intensity, the greater the current in the emitter. Since a resistor (10k) is connected to the emitter, the greater the light intensity, the greater the voltage Vout will be.....



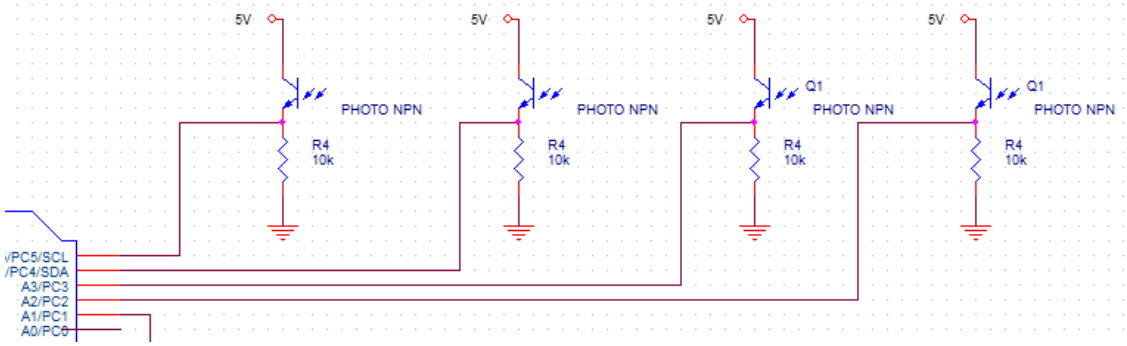
Program segment for the IR sensors:

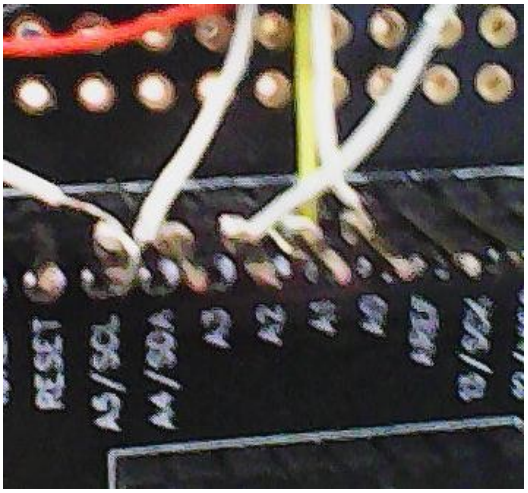
```
int IR1;
int IR2, IR3, IR4;
void setup() {
Serial.begin(9600);
}

void loop() {
IR1=analogRead(A2);
IR2=analogRead(A3);
IR3=analogRead(A4);
IR4=analogRead(A5);
Serial.print("IR1=");
Serial.println(IR1);
delay(1000);
Serial.print("IR2=");
Serial.println(IR2);
delay(1000);
Serial.print("IR3=");
Serial.println(IR3);
delay(1000);
Serial.print("IR4=");
Serial.println(IR4);
delay(1000);
}
```

IR Sensor Connections

We connected 4 IR receivers to the four sides of the cart. The sensors have 3 connections. One connection to the voltage output, a second connection to ground and a third connection to the Arduino Uno. While the program is running, a check is performed as to in which direction there is light. If the direction is not the front direction, that is – one of the rear/left/right sensors detects light, this is a sign that the shopper is on one of the sides of the cart and therefore it turns until the front sensor detects light – this means that the cart is behind the shopper. When the front sensor is active the cart drives forward as long as the distance from the shopper is not too short. The front sensor is connected to pin A2, the left to pin A3, the right to pin A5 and the rear to pin A4.

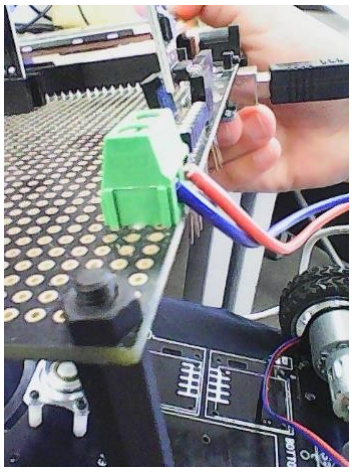


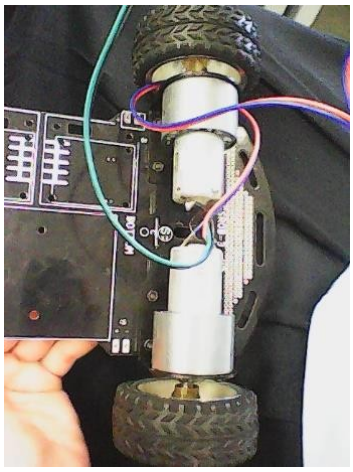


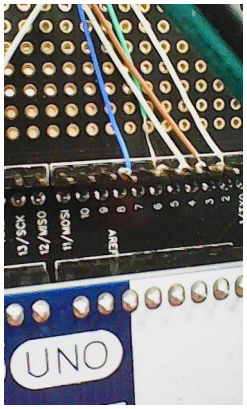


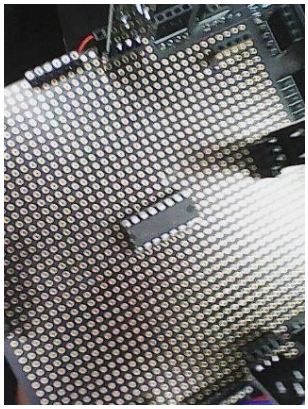
DC Motor Connected to the L293D Driver

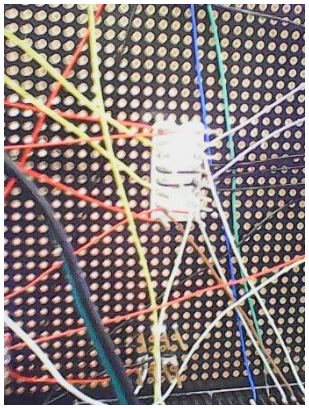
We connected the DC motors to a motor driver and connected it to the Arduino Uno. The driver supplies the currents and voltages required for operating the motor and also for determining the rotation direction. From the right motor we connected wires to pins 11 and 14 of the driver, which are parallel to pins 10 and 15, which we connected to pins. The left motor we connected to pins 3 and 6, which are parallel to pins 2 and 7, which we connected to the Arduino Uno at pins 3 and A1.











A DC motor contains 2 main parts:

a. **Motor stator** – the stator is a 'static' (fixed) part that does not move, and on it are permanent magnetic poles.

b. **Motor rotor** – the rotor is the part that rotates. On the rotor is the coil with changing magnetic poles.

Using Figure 3 and Figure 4, we can understand how the rotation of the motor takes place:

DC Motor Operation Principle

When the direction of the current through the coil of the rotor is as drawn, an attractive force is created between the poles and the rotor will move clockwise.

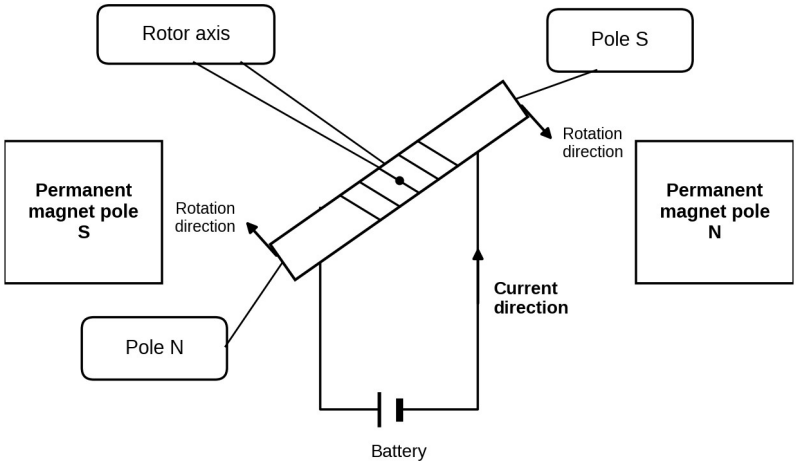


Figure 3

When the rotor reaches a balanced position it will stop. But if we reverse the polarity of the battery's voltage, the current through the coil of the rotor will change direction, the polarity of the electromagnet of the rotor will switch (as shown in drawing 4) and we will get repulsion between the poles.

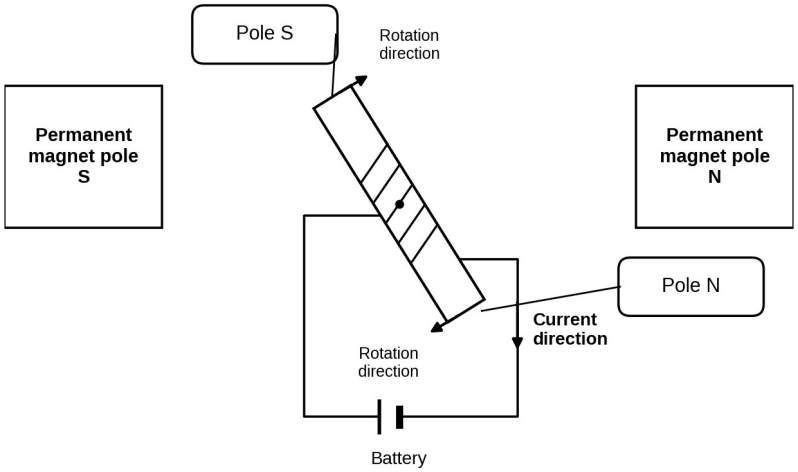
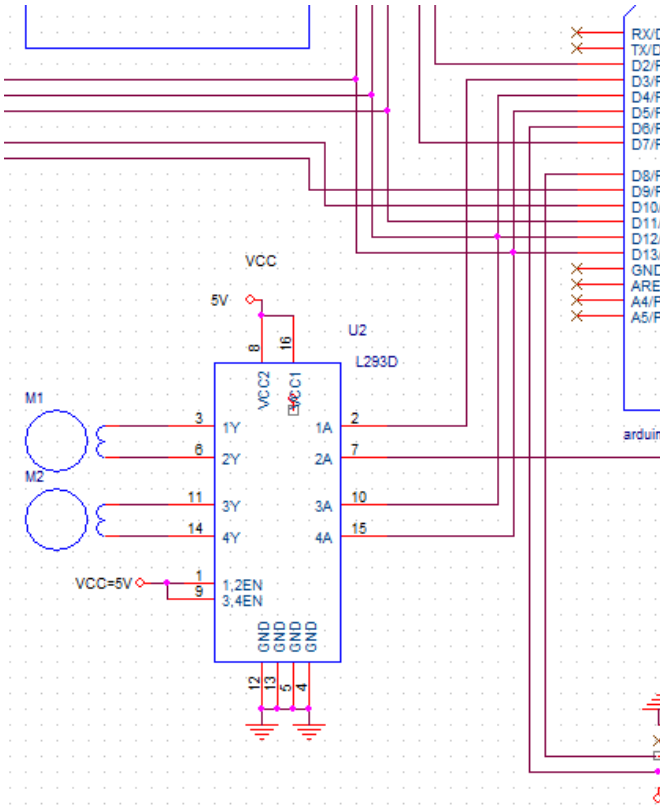


Figure 4

We see that reversing the direction of the current in the rotor's coil will cause a reversal of the magnetic poles in the rotor and the continuation of the motor's rotation. But the battery has voltage terminals with fixed polarity, so how do we reverse the direction of the current?

For this purpose a commutator is built. The commutator reverses the polarity of the voltage on the coil with every rotation of 180 degrees. In practice the switching is performed by a commutator built from electrical contacts similar to brushes.

DC Motor Connections



Motor Driver

The motor control system consists of:

An L293D switching component made of an H-bridge, whose role is to supply the voltages and currents required for the motors and to determine the rotation direction of the motor.

Pins D5, D6 produce a square pulse with PWR modulation.

Pins D3, D8 determine the rotation direction of the motors.

D293 H-Bridge Driver Circuit

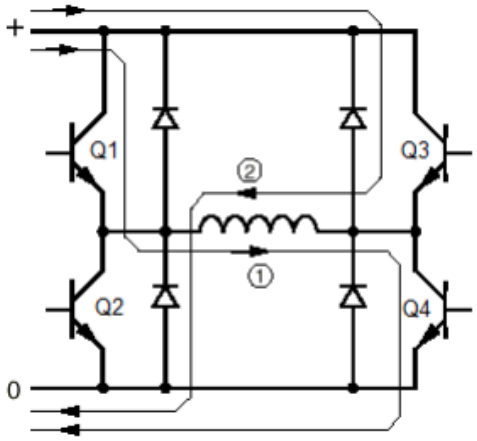
Features:

- Maximum current 600mA
- Momentary current 1.5A
- Operating voltage range 4.5V to 36V
- Protection diodes on every output against inductive load

Role of the component

The role of this component is to operate a DC motor in two directions, by switching internal transistors according to digital control.

The outputs are protected by internal diodes whose role is to prevent the voltage from reaching high values caused by the inductance of the motor (according to Lenz's law).



When Q1 and Q4 conduct, current flows in path 1 and the motor will rotate in one direction, while when transistors Q2 and Q3 conduct, current flows in path 3 and the motor will rotate in the other direction.

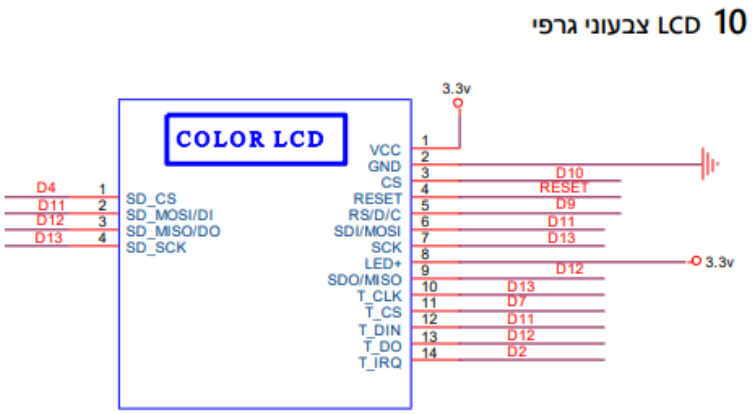
When we operate the motor using PWM modulation, we make sure, for example, that transistor Q1 conducts constantly and Q4 conducts according to the PWM signal. When Q4 conducts, current flows according to path 1, and when Q4 does not conduct, a freewheeling current flows in path 2 through Q1 and through the diode.

Program that operates the motors:

```
//motor1
#define PWM1 3
#define DIR1 A1
//motor2
#define PWM2 4
#define DIR2 5
Void motors (int speed1, int speed2) // motors
{
if (speed1 >= 0) digitalWrite (DIR1, LOW);
else digitalWrite (DIR1, HIGH);
if (speed2 >= 0) digitalWrite (DIR2, LOW);
else digitalWrite (DIR2, HIGH);
analogWrite (PWM1, speed1);
analogWrite (PWM2, speed2);
}
void setup() {
pinMode (DIR1, OUTPUT); // motors
pinMode (DIR2, OUTPUT); // motors
void loop () {
motors (200,200);
delay (2000);
motors (0,0);
delay (1000);
motors (200,-200);
delay(2000);
motors (0,0);
delay (1000);
motors (-200,200);
delay(2000);
motors (0,0);
delay (1000);
}
```

Color Graphic LCD

We used the screen in order to give instructions to the customer and so that he can enter his commands into the program through the screen, which receives the press from him and sends it to the program.



A color graphic LCD with a resolution of 320*240 RGB pixels.

Each pixel receives a value of 16bit or 18bit that make up the color.

Data is sent by means of SPI serial communication, consisting of 4 pins: MISO, MOSI, CS, SCK.

(3 systems can be connected: display, touch screen, memory card).

And two more control pins: the RS pin for determining data or control, and RESET for reset.

Function of the pins:

Reset – initializes the display processor.

RS/AD – the pin that determines whether a control word is sent (pixel address) RS=0, or determines whether color data is sent RS=1.

SDI/MOSI – serial data input line.

SDO/MISO – serial data output line.

SCK – clock line that the external processor sends.

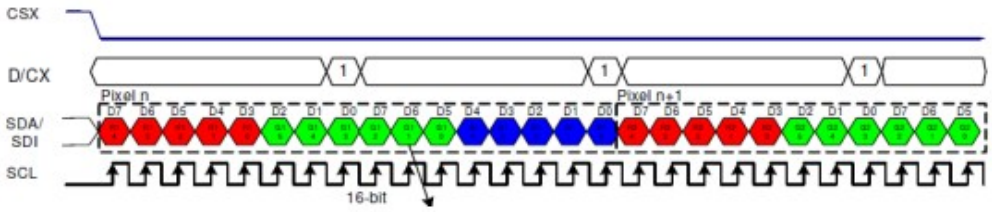
CS – communication enable pin; when communication begins, the pin goes down to 0.

Setting the Color of Each Pixel

Information about the color of the pixel can be sent by 16bit, and then two bytes of information are sent, or by 18bit, and then three bytes are sent, and each byte contains information about green, red or blue color.

16bit mode

Within the transmitted 16Bit word are all the colors, according to the following description:



- 5 bits – a component of red color.
- 6 bits – a component of green color.
- 5 bits – a component of blue color.

By combining all the colors, 65K colors can be obtained.

In order to reach each pixel, you must first write the address and then the data.

According to the diagram you can see that at the beginning of communication CS=0 and afterwards 16 bits are transmitted serially.

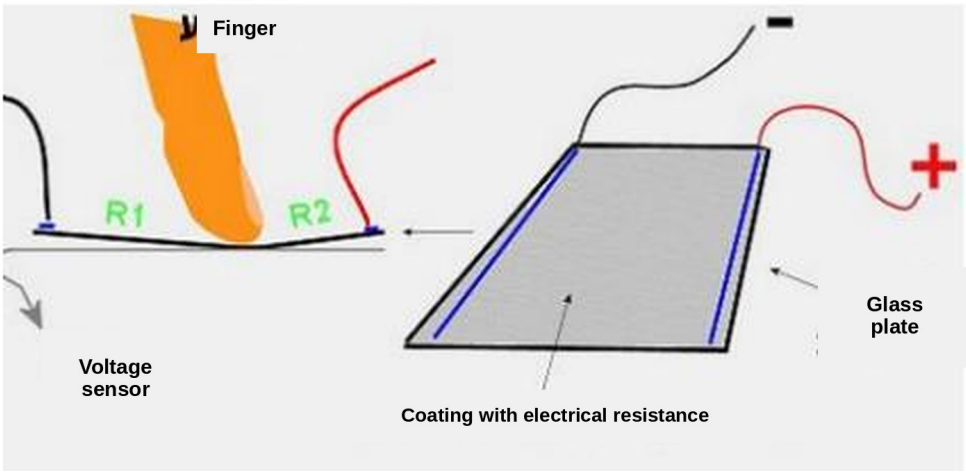
On each rising edge of the clock one bit is transferred according to the protocol. In order to determine whether the transmitted data will be an address or data, we change the logical state of RS.

Function name	Description	Parameters that the function receives or returns
`Lcd.begin()`	Initializes the communication between the screen and the controller.	
`Lcd.clrscr()`	Clears the screen.	Receives the desired clearing color.

TOUCH Screen



Operating principle



The screen is a resistance-coated surface covered with thin glass (as described in the diagram). When touching the screen, a division of the screen into two resistors R1 and R2 on the Y axis is actually performed. In order to create a division on the X axis as well, there is an additional surface, so that a division into two resistors is performed on this axis too.

A division into two resistors at every touch point enables a voltage division; the voltage will always be proportional to the touch.

Explanation of LCD Screen Functions

Function name	`void begin()`
Description – Display initialization	

`void fillScreen(uint16_t color)`
Description – Fills the entire screen with a color
Function receives – color – the screen color according to 16 bits (5bit RED, 6bit Green, 5bit Blue) or the capitalized name of the color.
Some of the colors:

`#define BLACK 0x0000`
`#define GRAY 0x8410`
`#define BLUE 0x001F`
`#define RED 0xF800`
`#define GREEN 0x07E0`
`#define CYAN 0x07FF`
`#define MAGENTA 0xF81F`
`#define PURPLE 0x7194`
`#define ORANGE 0xFE00`
`#define YELLOW 0xFFE0`
`#define WHITE 0xFFFF`

`Example: LcdTouch.fillScreen(RED);  // RED = 0xF800`

`void setRotation(uint8_t m)`
Description – Rotates the screen in 4 directions
Function receives – a value from 0 to 3

`void setTextSize(uint8_t fontSize)`
Description – Sets the font size
Function receives – a font size value of 1 and above (default 2)

`void setCursor (uint16_t x,uint16_t y)`
Description – Selects a position on the screen.
Function receives – X, Y values

`void setTextColor(uint16_t color)`
Description – Changes the text color.
Function receives – color – the screen color according to 16 bits

`void setTextColor(uint16_t color, uint16_t background)`
Description – Sets the text color and the text background (the background erases previous text)
Function receives – the text color and the background color of the text.
`Example: setTextColor(BLACK,RED);`

`void print (int num)`
Description – A function that prints the numeric value of the variable.
Function receives – a variable of 16 bits size
`Example: print (x);   // prints the value of x`

`void print (char *str)`
Description – A function that prints a string.
Function receives – a string.
`Example: print ("Press Keyboard to return");`

`void println (int num)`
Description – A function that prints the numeric value of the variable and moves down a line.
Function receives – a variable of 16 bits size

`void println (char *str)`
Description – A function that prints a string and moves down a line.
Function receives – a string.

Description – Writing a message on the screen, with setting of the position, font size, message color.
Function receives – message position x, y, a string, font size, message color.
`Example: print(40, 20, "hello", 2,RED);`

`void print(uint16_t x, uint16_t y, char *str, uint16_t fontSize, uint16_t fColor, uint16_t bColor)`
Description – Writing a message on the screen, with setting of the position, font size, message color and message background.
Function receives – message position x, y, a string, font size, message color, background color.
`Example: print(40, 20, "hello", 2,RED,BLACK);`

`void printheb(char *str)`
Description – A function that receives a string, intended for displaying on the screen in Hebrew.
Function receives – a Hebrew string
`Example: printheb("Hello");`

`void printhebln(char *str)`
Description – A function that prints a message in Hebrew on the screen and moves down a line.
Function receives – a Hebrew string

`void printheb(uint16_t x, uint16_t y, char *str, uint16_t fontSize, uint16_t fColor)`
Description – Writing a message in Hebrew on the screen, with setting of the position, font size, message color.
Function receives – message position x, y, a string to display, font size, message color.
`Example: printheb(10,20,"Hello",2,RED);`

`void printheb(uint16_t x, uint16_t y, char *str, uint16_t fontSize, uint16_t fColor, uint16_t bColor)`
Description – Writing a message in Hebrew on the screen, with setting of the position, font size, message color and background.
Function receives – message position x, y, a string to display, font size, message color, background color.
`Example: printheb(10,20,"Hello",2,RED,BLACK);`

`void drawPixel (int16_t x,int16_t y, uint16_t color)`
Description – Coloring a single pixel.
Function receives – the position of the pixel x, y, the color of the pixel.
`Example: drawPixel (100 ,150 , GREEN);`

`void drawHLine (int16_t x,int16_t y, int16_t w, uint16_t color)`
Description – Drawing a horizontal line
Function receives – the position where the line starts x, y, the length of the line, the color of the line.
`Example: drawHLine (20,40, 100, ORANG) ;`

`void drawVLine (int16_t x,int16_t y, int16_t h, uint16_t color)`
Description – Drawing a vertical line
Function receives – the position where the line starts x, y, the height of the line, the color of the line.
`Example: drawVLine (30,50, 100, BLUE) ;`

`void drawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t color)`
Description – Drawing a line
Function receives – the values of the start of the line x0 y0, the end of the line x1 y1, the color of the line.

`void drawRect (int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color)`
Description – Drawing a rectangle outline in color.
Function receives – the x, y values of the upper-left corner of the rectangle, length and width, the color of the outline.
`Example: drawLine(10, 10, 100, 100, BLACK);`

`void drawRoundRect(int16_t x, int16_t y, int16_t w,int16_t h, int16_t r, uint16_t color)`
Description – Drawing a rectangle outline in color, with rounded corners
Function receives – the x, y values of the upper-left corner of the rectangle, length and width, the degree of corner rounding, the color of the outline.
`Example: drawRoundRect(10, 10, 50,100, 5, RED)`

`void drawTriangle(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color)`
Description – Drawing a triangle outline.
Function receives – the x, y values of the triangle's vertices, the color of the triangle's outline.
`Example: drawTriangle(10, 100, 100, 100,20 ,20, RED);`

`void drawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color)`
Description – Drawing a circle outline.
Function receives – the position of the circle's center, the radius of the circle, color.
`Example: drawCircle(100, 100, 20 , BLACK);`

`void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color)`
Description – Drawing a rectangle filled with a chosen color.
Function receives – the x, y values of the upper-left corner of the rectangle, length and width, the color of the rectangle.
`Example: fillRect (10, 10, 100, 100, BLACK) ;`

`void fillRoundRect(int16_t x, int16_t y, int16_t w, int16_t h, int16_t r, uint16_t color)`
Description – Drawing a rectangle filled with a chosen color, with rounded corners
Function receives – the x, y values of the upper-left corner of the rectangle, length and width, the degree of corner rounding, color.
`Example: fillRoundRect(10, 10, 50,100, 5, RED);`

`void fillTriangle ( int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color)`
Description – Drawing a triangle filled with a chosen color.
Function receives – the x, y values of the triangle's vertices, color.
`Example: drawTriangle(10, 100, 100, 100,20 ,20, RED)`

`void fillCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color)`
Description – Drawing a circle filled with a chosen color.
Function receives – the position of the circle's center, the radius of the circle, color.

`void drawButton(int8_t NumButton, int16_t x, int16_t y, int16_t w, int16_t h, int16_t r, uint16_t Color, uint16_t textcolor, char *label, int8_t textsize)`
Description – Drawing a button on the screen with text.
Function receives – the button number, position x,y, height – h, width – w, corner rounding – r, the color of the button, the color of the text, the text, the font size
`Example: drawButton(1, 20, 20, 90, 50, 10, RED, WHITE, "EXAM1", 2) ;`

`int8_t ButtonTouch(int16_t x, int16_t y)`
Description – Checks which button number was pressed according to position.
Function receives – the position of the press x, y
Function returns – the button number
`Example: int ButtonNum = LcdTouch.ButtonTouch(LcdTouch.xTouch, LcdTouch.yTouch);`

`bool touched()`
Description – Checks whether there is a press
Function returns – pressed or not

`void readTouch()`
Description – Reading the position of the press and storing them in the class's two variables: xTouch, yTouch

`Example:`
`LcdTouch.readTouch();`
`x = LcdTouch.xTouch;`
`y = LcdTouch.yTouch;`

Documentation of the sequence of operations by screens:

Screen number 1:



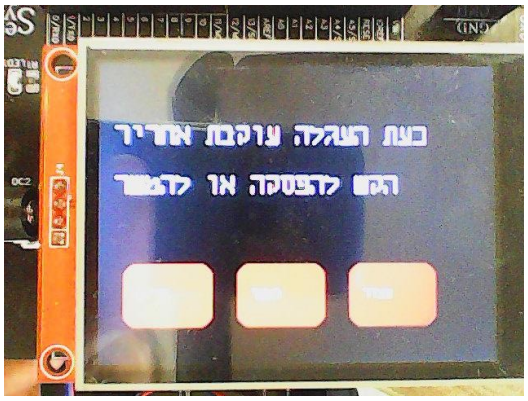
In order for the program to start operating, the customer presses the Start button on the first screen.

The actions that occur when pressing the Start key:

The operation of the IR sensors begins to operate (detailed on the page about the IR sensors); within this operation there are several references to the program of the distance sensor (detailed on the page about the distance sensor), so that the cart will not drive after the shopper and get stuck on an obstacle that will pass between it and the shopper.

The screen moves to screen number 2.

Screen number 2:



Three buttons appear on the screen:

First button – Stop, by pressing this button the program stops and the screen remains.

Second button – Continue, by pressing this button the program continues and the screen remains.

Third button – Finish, by pressing this button the program stops finally and the screen changes to the third screen.

Screen number 3:



On the screen there is one button – Back, whose press leads to the first screen.

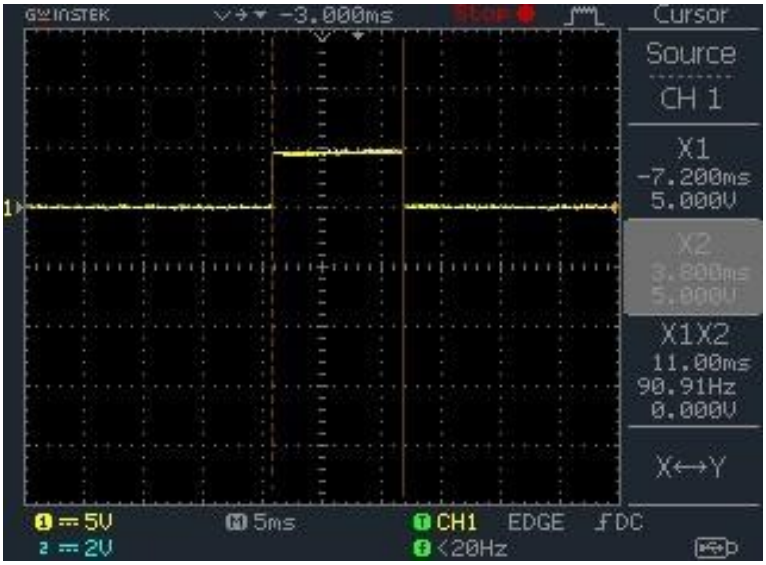
Measurements on the screen later in the book.

Screens program:

```
void screenMain() {
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(70, 50, "Welcome!", 3, WHITE);
LcdTouch.printheb(0, 90, "Press to start shopping", 3, WHITE);
LcdTouch.drawButton(1, 20, 160, 80, 50, 5, RED, WHITE, "Start", 1);
}
void screen2() {
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(1, 50, "The cart is now following you", 2, WHITE);
LcdTouch.printheb(1, 90, "Press to pause or continue", 2, WHITE);
LcdTouch.drawButton(2, 200, 160, 70, 50, 10, RED, WHITE, "Stop", 1);
LcdTouch.drawButton(3, 110, 160, 70, 50, 10, RED, WHITE, "Continue", 1);
LcdTouch.drawButton(4, 20, 160, 70, 50, 10, RED, WHITE, "Finish", 1);
}
void screen3() {
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(20, 50, "Have a nice day!", 3, WHITE);
LcdTouch.printheb(0, 90, "Press to return to the main screen", 2, WHITE);
LcdTouch.drawButton(5, 20, 160, 80, 50, 10, RED, WHITE, "Back", 1);
}
```

Measurements / Distance Sensor

The yellow wave describes the ECHO pin of the distance sensor. The time in seconds is the width of the pulse (in units of microseconds) divided by 58. In the first picture the width of the pulse is 11 milliseconds, that is, 11 to the power of 3 microseconds. Dividing by 58 gives approximately 189.



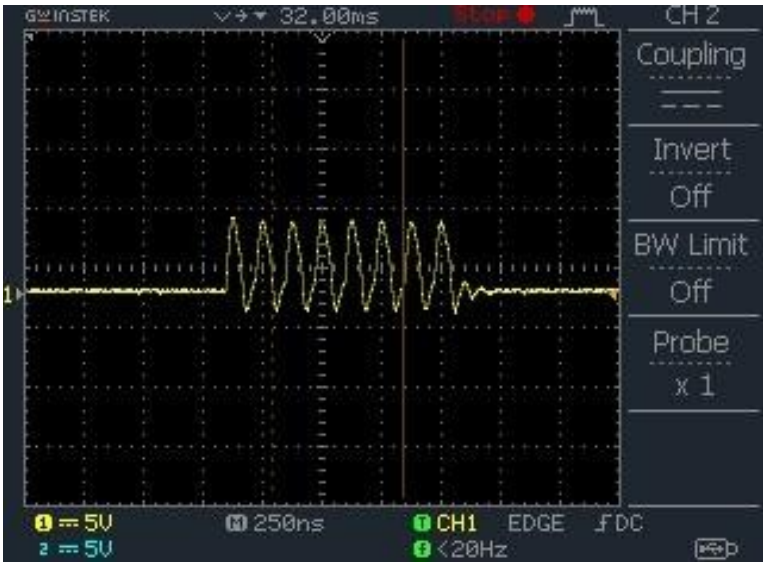
In this picture the wave of the ECHO pin is displayed together with the wave of the TRIGER.

In the wave of the TRIGER you can see the pulse that gives voltage in order to activate the sensor.

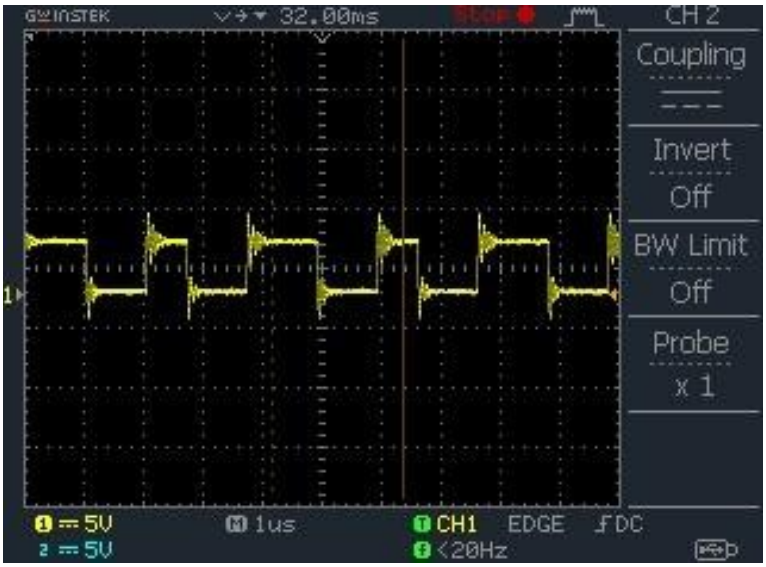


Measurements / Screen

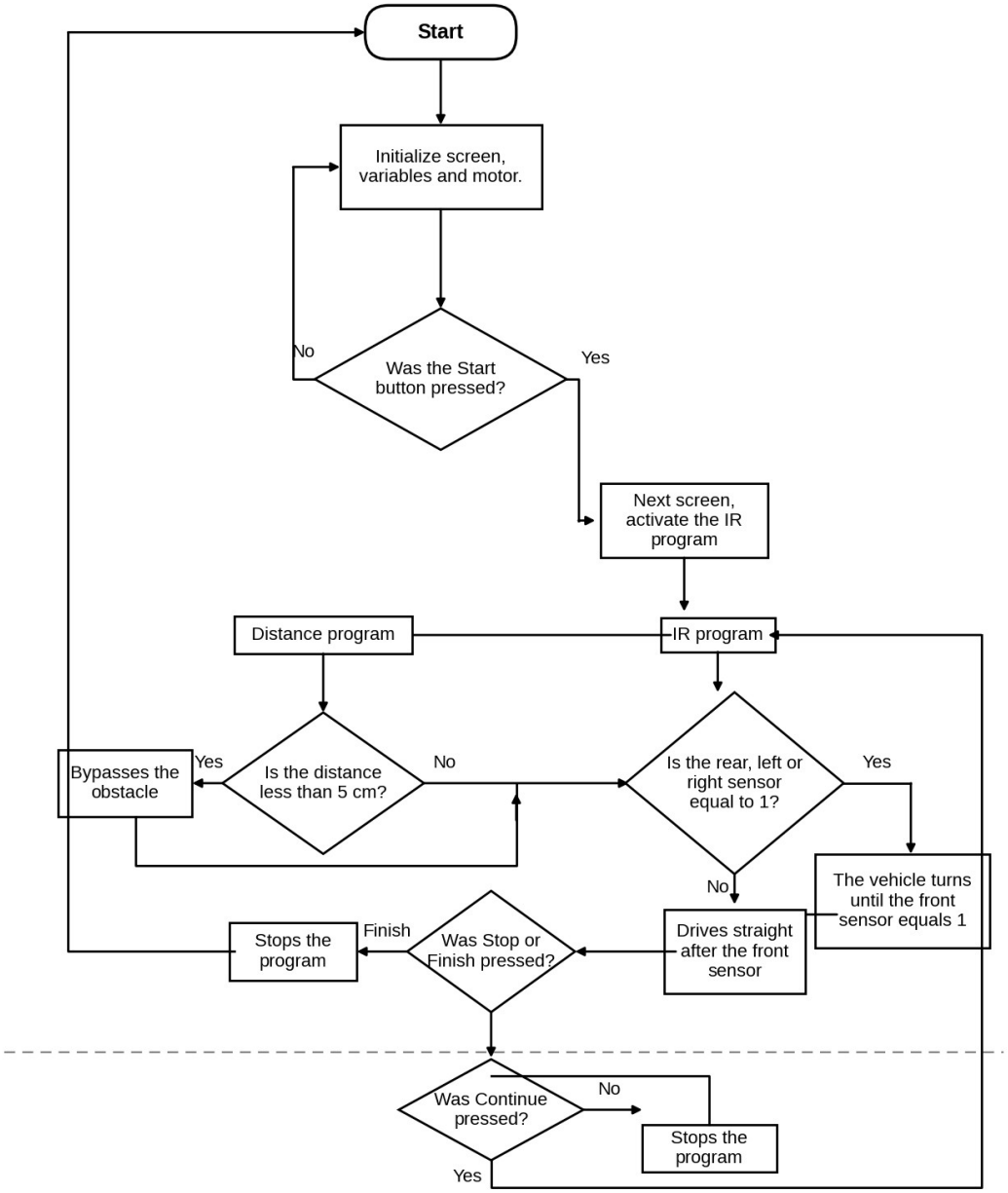
The yellow wave describes the SCK of the LCD screen. When the screen changes, there is a wave of 8 bits – the wave in the picture.



In this picture, the wave describes the MOSI of the screen.



Flowchart


Start
Initialize screen, variables and motor.
Was the Start button pressed?
No
Yes
Next screen, activate the IR program
Distance program
IR program
Is the distance less than 5 cm?
Yes
No
Bypasses the obstacle
Is the rear, left or right sensor equal to 1?
Yes
No
The vehicle turns until the front sensor equals 1
Drives straight after the front sensor
Was Stop or Finish pressed?
Finish
Stops the program
Was Continue pressed?
No
Stops the program
Yes

Program Segment

```
int IR1, IR2left, IR3revers, IR4right;// tracking
int trig=8,echo=6;// distance
int time,distance;// distance
int a=0;
//motor1
#define PWM1 3
#define DIR1 A1
//motor2
#define PWM2 4
#define DIR2 5
#include "ArdRTOS.h" //two actions in parallel
#include "TFT9341Touch.h"// screen
tft9341touch LcdTouch(10, 9, 7, 2);
bool lock = false;
void setup() {
Serial.begin(9600);
LcdTouch.begin();
LcdTouch.clearButton();
LcdTouch.setRotation(0);
LcdTouch.setTextSize(2);
LcdTouch.setCursor(40, 40);
LcdTouch.set(3780, 372, 489, 3811); //Calibration
screenMain();
Serial.println("after screenMain");
pinMode(DIR1, OUTPUT); // motors
pinMode(DIR2, OUTPUT); // motors
motors(0,0); // motors
pinMode(trig,OUTPUT);// output - distance
pinMode(echo,INPUT);// voltage input - distance
}
void screenMain() //first screen
{
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(70, 50, "Welcome!", 3, WHITE);
LcdTouch.printheb(0, 90, "Press to start shopping", 3, WHITE);
LcdTouch.drawButton(1, 20, 160, 80, 50, 5, RED, WHITE, "Start", 1);
uint16_t x, y;
int8_t ButtonNum;
while(1)
{
while (!LcdTouch.touched());
LcdTouch.readTouch();
x = LcdTouch.xTouch;
y = LcdTouch.yTouch;
int8_t ButtonNum = LcdTouch.ButtonTouch(x, y);
if (ButtonNum == 1) {
break;
}
}
}
void screen2() //second screen
{
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(1, 50, "The cart is now following you", 2, WHITE);
LcdTouch.printheb(1, 90, "Press to pause or continue", 2, WHITE);
LcdTouch.drawButton(2, 200, 160, 70, 50, 10, RED, WHITE, "Stop", 1);
LcdTouch.drawButton(3, 110, 160, 70, 50, 10, RED, WHITE, "Continue", 1);
LcdTouch.drawButton(4, 20, 160, 70, 50, 10, RED, WHITE, "Finish", 1);
```

```
uint16_t x, y;
int8_t ButtonNum;
while(1)
{
while (!LcdTouch.touched()) {
IRstart();
// if (lock){return;}
}
if (LcdTouch.touched()) {
LcdTouch.readTouch();
x = LcdTouch.xTouch;
y = LcdTouch.yTouch;
ButtonNum = LcdTouch.ButtonTouch(x, y);
}
if (ButtonNum == 4||ButtonNum==2) {
break;
}
}
if(ButtonNum==4){screen3();}
else if (ButtonNum == 2) {
stop();
while(1)
{
while (!LcdTouch.touched());
if (LcdTouch.touched()) {
LcdTouch.readTouch();
x = LcdTouch.xTouch;
y = LcdTouch.yTouch;
ButtonNum = LcdTouch.ButtonTouch(x, y);
}
if (ButtonNum == 3)
{break;}
}
IRstart();
}
}
void screen3() // third screen
{
stop();
LcdTouch.clearButton();
LcdTouch.fillScreen(BLACK);
LcdTouch.printheb(20, 50, "Have a nice day!", 3, WHITE);
LcdTouch.printheb(0, 90, "Press to return to the main screen", 2, WHITE);
LcdTouch.drawButton(5, 20, 160, 80, 50, 10, RED, WHITE, "Back", 1);
uint16_t x, y;
int8_t ButtonNum;
while(1)
{
while (!LcdTouch.touched());
if (LcdTouch.touched()) {
LcdTouch.readTouch();
x = LcdTouch.xTouch;
y = LcdTouch.yTouch;
ButtonNum = LcdTouch.ButtonTouch(x, y);
}
if (ButtonNum == 5){break;}
}
screenMain();
}
void motors(int speed1, int speed2) // motors
{
if (speed1 >= 0) digitalWrite(DIR1, LOW);
```

```
else digitalWrite(DIR1, HIGH);
if (speed2 >= 0) digitalWrite(DIR2, LOW);
else digitalWrite(DIR2, HIGH);
analogWrite(PWM1, speed1);
analogWrite(PWM2, speed2);
}
void DISTANCE() //distance sensor
{
distance=0;
time=0;
digitalWrite(trig,HIGH);//trigger
delayMicroseconds(10);
digitalWrite(trig,LOW);//trigger
while(digitalRead(echo)==0);//echo
time=micros();//micro second time
while(digitalRead(echo)==1);//echo
time=micros()-time;
distance=time/58;
Serial.println("distance:");
Serial.println(distance);
delay(1500);
if (distance<=25&& distance>0)
{
IR1 = digitalRead(A2);// front sensor
while(IR1==0){motors(0,0);if (LcdTouch.touched()){a=1;return;}IR1 =
digitalRead(A2);}
motors(200,200);
delay(2000);
motors(0,0);
delay(1000);
motors(200,-200);
delay(2000);
motors(0,0);
delay(1000);
motors(-200,200);
delay(2000);
motors(0,0);
delay(1000);
motors(200,200);
}
}
void IRstart() {
IR1 = digitalRead(A2);// front sensor
IR2left = digitalRead (A3);// left sensor
IR3revers = digitalRead (A4);// rear sensor
IR4right = digitalRead (A5);// right sensor
Serial.println ("IR1 :");
Serial.println (IR1);
Serial.println ("IR2left :");
Serial.println (IR2left);
Serial.println ("IR3revers :");
Serial.println (IR3revers);
Serial.println ("IR4right :");
Serial.println (IR4right);
Delay (2000);
if (IR2left==0) {
while (IR1==1 ) {motors(-200, 200);delay(50);IR1 =
digitalRead(A2);Serial.println(IR1);};
}
else if (IR3revers==0) {
while (IR1==1) {motors(-200, 200);delay(50);IR1 =
digitalRead(A2);Serial.println(IR1);}
```

```
}
else if (IR4right==0) {
while (IR1==1) { motors(200, -200);delay(50);IR1 =
digitalRead(A2);Serial.println(IR1);}
}
while (IR1==0){
DISTANCE();
if(a==1){return;}
motors(-100,-100);
Serial.println(IR1);
IR1 = digitalRead(A2);
}
}
void stop() {
motors(0, 0);
}
void loop() {
screen2();
}
```

Log

30/10/23
We learned about binary numbers

6/11/23
We learned about touch screens and how to write them.

8/11/23
We wrote the screens, and there was a problem – we did not initialize the buttons and therefore the button remained, and every time we pressed return to the first screen, we initialized the buttons.

13/11/23
We tried to operate the screen – there was a problem that only the first button worked and the rest of the buttons did not operate. The problem was solved when

15/11/23
We learned about the distance sensor, started writing its program and started assembling the cart.

22/11/23
We performed the soldering on the Arduino.

18/12/23
We finished the program of the distance sensor, checked that it works properly and connected wires to the motor.

20/12/23
We connected wires to the motor driver

25/12/23
We connected wires according to the electrical schematic and thought about a solution to the problem that the IR sensor does not work if something hides it and it can lose the shopper. The solution – to program the cart so that in such a case, where it does not see the sensor, it will continue in the same direction in which it was going until it sees the sensor. In addition, we learned about serial communication.

1/1/24
We connected wires from the driver to the motor's mount.

3/1/24
We wrote in the project book about the distance sensor, saw the structure of the project book, read about the IR sensor

8/1/24
We started the program of the wheels and started writing the program of the IR sensors and the main program, and fixed wires that were not connected in the right place.

22/1/24
We continued with the program of the wheels and exported it. The program worked but only on one wheel, so we checked why, and we want to check whether it is because both pins are connected.

24/1/24
We learned about serial communication and tried to find the problem in the wheels

29/1/24
We saw that we had not connected 2 wires to the driver, so we connected them, but the problem was not solved, so we tried to solve it and did not succeed but on the contrary – we saw that the program does not really work even on one wheel. We started making a main program.

31/1/24
We found the problem – the Arduino and the motor of the wheel were faulty. We replaced them and the program worked. We did an experiment with IR sensors and the program worked; the program wrote whether the laser is transmitted to it according to the level of blockages that we marked between them.

5/2/2024

We started connecting the components of the IR sensors to the Arduino Leonardo. By mistake we connected the receiver to the Leonardo, so we took it out and connected the LED.

7/2/24

We tried to operate the LED and it read only 0, and then we saw that it was connected to ground without a resistor because it was shorted, so we canceled the connection and then the LED worked. Afterwards it stopped working again and we discovered that the LED was faulty. We started writing the tracking program in the main program.

12/2/24

We continued the main program – we started connecting between the screens and the IR sensor

14/2/24

We progressed with the main program and there is a problem: only the first button works. In addition, we started connecting the three additional mounts for the IR receivers.

19/2/24

We continued connecting the mounts of the IR. We wrote in the main program what it does with the distance sensor and it did not work for us. Why?

26/2/24

The teacher found the solution to only the first button working: we saw that it may be that when we pressed, the program was inside the IF, and therefore we do a long press so that it exits the IF. There was a problem that the sensor did not display real values, and the problem was that the driver made voltage jumps, so we connected a capacitor.

6/3/24

We progressed with the two tasks in parallel and started measurements, but it did not photograph for us.

18/3/24

We finished writing the main program and connecting the IR sensors. The problems to solve:

1. When pressing the Stop button, the screen goes black
2. IR sensors display incorrect values – the front and left sensor
3. The actions do not run in parallel as they are supposed to.

27/3/24

We moved all the button presses from the loop to the screens. Here the problem with the Stop button was solved. We progressed in the project book.

1/4/24

We discovered that the IR sensors were reversed because in Moti Meir it was written reversed. We progressed in the project book.

3/4/24

We performed measurements on the distance sensor and on the screen. The IR sensors are still wrong – probably faulty. We added the lock check in additional places and not only at the beginning of the loop.

1/5/24

A problem that we found is: the program that runs 2 actions simultaneously runs them only once, and therefore the distance program repeats only once. In addition, we found sensors that will be a substitute for the IR sensors that do not work, we performed tests and saw that they work, and we started replacing the sensors.

6/5/24

We finished replacing the IR sensors and saw that they work. In addition, we canceled the simultaneous operation of the two actions and made references to the distance function from several places in the program.

13/5/24

We operated the project after the changes; the motors gave problems and therefore we replaced one of the motors.

Faults

- The IR sensors displayed incorrect values. After checking the connections and the program, we saw that the problem was in the sensors and not in the program. When we replaced the sensors the problem was solved.
- At a certain stage we saw that only the first button works. The problem was that we pressed the next buttons with a short press, and since there is an IF that the program could enter, the press was not registered as long as the program was in the IF. The solution is to press a slightly longer press.
- We used a program that runs two actions simultaneously, but it ran only once one of the programs, and the other one in a loop. The reason for this was that one of the programs was referenced from within a screen, and the program that runs the actions simultaneously does not work with screens. In order to solve the problem, we ran the programs simultaneously in the form of a reference to the second function in several places in the first function.
- The wheels did not drive at the same rate, the problem was in the motor.
- When we pressed the Stop button, the screen went black. We moved all the presses from the loop to the screens and the problem was solved – the problem was that it registered the press not inside a WHILE, and therefore it was necessary to press at a specific second. When we moved to the screens we put everything inside a WHILE.
- When we pressed the button that moves from the first screen to the second, the screens appeared on top of each other, and we discovered that we forgot to clear the first screen on the transition to the second screen.

SPI Communication

Serial synchronous communication (Serial Peripheral Interface), between two devices.

Mater-Slave When the Master supplies a clk to the system, and the Slave sends/receives data according to this clock.

The SPI system is a full duplex system (Full Duplex), that is, sending data and receiving data (through 2 wires) in it simultaneously.

In order to use SPI communication, four lines are needed: 2 control lines and 2 data lines:

MOSI (Master Out Slave In) – data is sent from the Master to the Slave.

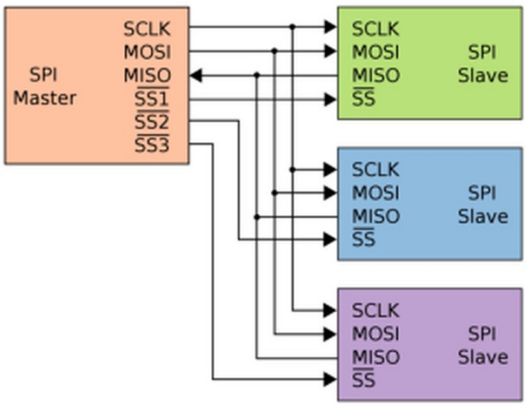
MISO (Master In Slave Out) – data is sent from the Slave to the Master.

SCLK – serial clock – determines the timing of data transfer between the Master and the Slave.

SS (Slave Select) – this line is active low and provides control for starting the communication operation with the Slave.

13.1.1 Physical connection

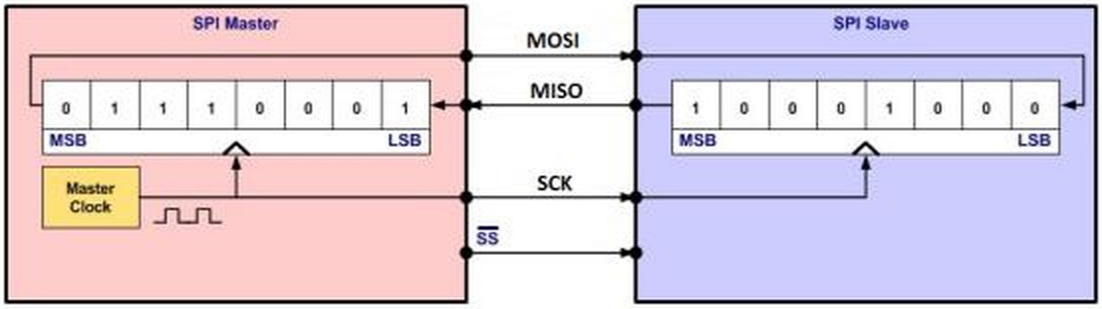
In this communication it is possible to connect several Slave devices in parallel, when the Master lowers the SS pin of only one Slave.



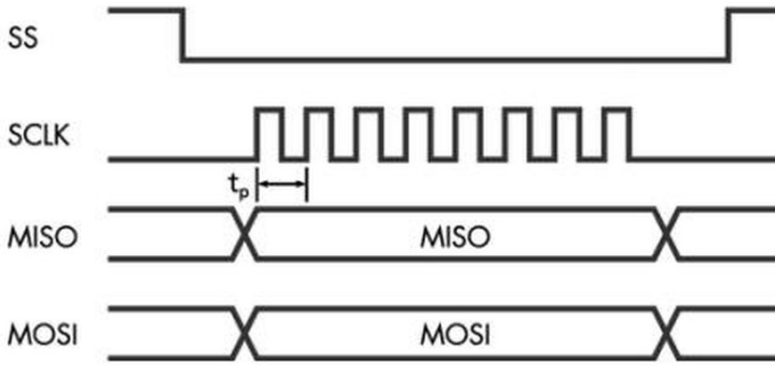
Communication Protocol

The data is transferred simultaneously in both directions on every clock rise, by means of a ring shift register.

In the following example: after 8 clock cycles, data is exchanged between the Master and the Slave.



Sending a control word of 8 bits



At the beginning of the communication, the SS pin goes down to 0. Afterwards the data is transferred from the Master through the MOSI pin. In return, the Master can read the data from the Slave through the MISO pin. At the end of the communication the SS pin returns to logical 1.

tp – clock cycle time.

Summary

Working on the project was challenging, demanding and in-depth. Familiarity with the theoretical material turned into practical doing that required understanding and thinking. At times it was difficult, but solving the problems and obstacles led to great satisfaction. We deepened the knowledge we acquired within the track and learned about various components and their use. In addition, we acquired technical tools and knowledge regarding the physical connection of the components to the controllers. Teamwork contributed greatly to us, as each of us learns her strengths separately and the abilities together. The great investment gave us learning and work habits that will help us in every area of life. In addition, we gained an understanding of the broad world of electronics engineering and the ability to understand related terms and concepts.